The MABEL Teleoperation System: Apple Vision Pro → Whole-Body Humanoid Retargeting, with Tip-Safe Locomotion, Fail-Safe Engagement, Clutch Re-Indexing, and Self-Collision Avoidance

MABEL Simulation Project*

July 11, 2026

Abstract

This document is the system-level record of MABEL's first-person teleoperation: an operator wearing an Apple Vision Pro (AVP) drives a mobile bimanual humanoid (swerve base, vertical lift, leaning torso, 3-DoF neck, two 7-DoF arms, two 17-DoF ORCA hands) so that, ideally, "you are the robot the moment you put on the headset." We specify the pipeline as four swappable layers — (i) a tip-over base *motion model*, (ii) a *whole-body controller* that engages torso/base/lift by a bounding-box rule, (iii) *decoupled* hand and arm *retargeting*, and (iv) a *safety supervisor* for tracking loss — plus two intuitiveness/safety features added on top: a two-handed-fist *clutch* for workspace re-indexing and an *arm self-collision* barrier. All robot-side quantities are computed from forward kinematics on the robot model (no privileged simulator state), so the same code runs in simulation and on hardware. We report measured frequencies, latencies, tracking accuracy, and safety behaviour from a reproducible headless harness (`scripts/tools/teleop_benchmark.py`).

Keywords: teleoperation, humanoid retargeting, Apple Vision Pro, whole-body control, control barrier function, self-collision avoidance, tip-over stability, MuJoCo.

1 Overview and architecture

The publisher `scripts/vision_pro_stream.py` broadcasts, each frame, the AVP head pose, body odometry and heading, head height, and per-hand 21-joint skeletons (plus elbows and a thumb–index pinch scalar) over ZMQ. The consumer `scripts/retarget_teleop.py` maps these to robot actuator targets and steps a MuJoCo model of the full robot. The data flow is

$$\text{AVP stream} \rightarrow \underbrace{\text{hand / arm / neck retarget}}_{\text{Layer 3}} \rightarrow \underbrace{\text{whole-body cascade}}_{\text{Layer 2}} \rightarrow \underbrace{\text{base motion model}}_{\text{Layer 1}} \rightarrow \text{gravity-compensated physics},$$

*Companion notes (do not duplicate their derivations): base motion (`motion_model.pdf`), whole-body IK (`whole_body_control.pdf`), arm control (`arm_control.pdf`), self-collision (`self_collision.pdf`), compliance (`compliance_study.pdf`).

with the *safety supervisor* (Layer 4) and the *clutch* acting across all layers. Each layer is a separate class with a narrow interface so it can be replaced without touching the others: `BaseMotionModel` (Layer 1), `WholeBodyController` (Layer 2, with pluggable `motion_model` and a `base_filter` control-barrier hook), `HandOptimizer`/`ArmRetargeter` (Layer 3), `EngagementSupervisor` and `ClutchController` (Layer 4), and `ArmCollisionProjector` (the arm barrier). The control loop runs the retargeting once per VR frame and free-runs the physics at the model timestep between frames (Sec. 3).

2 Method

2.1 Frames and the shared-world map

Operator points live in the AVP world frame (Y-up). They are carried to the robot world by a fixed rotation $A = \text{AVP_TO_ROBOT}$ (mapping forward→front, up→up, right→right) anchored *once* at the first frame by the pair $(p_0^{\text{avp}}, p_0^{\text{rob}}) =$ (operator head, robot head via FK):

$$W(p) = p_0^{\text{rob}} + s\,A\,\big(p - p_0^{\text{avp}}\big), \qquad s = \text{REACH_SCALE}. \tag{1}$$

Because the anchor is a *world* reference (not head-relative), the robot tracks the operator's global translation *and* heading: walking moves the base, turning the body turns the robot. The scalar s = robot reach/human reach is auto-calibrated at startup (operator extends both arms forward, presses Enter) and scales *only the arm* targets; the hand optimizer is fully decoupled with its own per-finger scales.

2.2 Hand retargeting (Layer 3a)

Each finger joint is matched in a canonical palm frame (forward = down the middle finger, side = index→pinky, normal = palm-out) built independently from the operator anatomy and from the ORCA geometry, so corresponding directions align by construction. A small SLSQP problem (cosine alignment + relative fingertip distances + smoothness + regularization) over the 16 finger DoFs is solved per hand; the thumb–index relative term is what lets the robot fingertips *touch* when the operator's do. (This SLSQP optimizer remains the desktop-entry finger solver and still supplies the canonical robot palm frame to the arm retargeter; on the *live app path* the fingers now run the webcam-parity anatomical decomposition instead — Sec. 4.)

2.3 Arm retargeting (Layer 3b)

The arm uses the *same* cosine philosophy lifted to the 7-DoF chain on a scratch `MjData`: match the head→elbow and contact-point directions and the palm orientation axes, with a per-joint regularization that resolves the redundancy like a human arm. The target normal is anatomically normalized per hand (`geometry.palm_axes`): the operator natural frame's normal is chirality-mirrored (palmar on the right hand, dorsal on the left) while the robot palm frame's normal is flexion-anchored (palmar on both sides), so the LEFT operator normal is flipped to palmar. Two 2026-07-11 forensic lessons are baked into the regression probe (`server/tools/exp_arm_symmetry.py`: stream exact x-mirrored 21-joint skeletons, FK the solved arms, compare the palms under the chassis mirror): *(i)* judge symmetry in Cartesian palm space — joint-space $|q|$ symmetry cannot see a palm inversion, which hides in the Arm 5/Arm 7/ORCA-wrist redundancy split; *(ii)* the probe's palmar axis must be anchored to the finger-*curl* (flexion) direction, never an index→pinky cross product — the left

ORCA palm's mirrored mount geometry makes the anatomical construction report a physically-inverted palm as symmetric. Measured with the physical metric: with the flip, palms mirror to 0.1° normal / 3.5 mm position; without it the left palm solves physically inverted (178.6°). Separately, on headset frames the wire-derived `forearm_arm` elbows steer only the base-yaw estimator and are excluded from the arm IK's elbow-direction cosine (ARKit's forearm joint is a hand-anchor extrapolation, not a tracked elbow; feeding it to the IK was the "left arm twisted" posture defect). The position task tracks a *contact point* a fixed distance ahead of the ORCA wrist along the palm (so bimanual fingertip contact maps to a single robot point, scale-invariantly). Two anatomy-driven weightings make the solution natural: the shoulder-roll joint is held so the forearm carries the roll, and the wrist-yaw joint (Arm 7) — which a human barely articulates (radial/ulnar deviation $\approx \pm 25°$) — is hard-bounded to rest ± 0.45 rad and regularized, forcing any hand flip onto the forearm (Arm 5) and wrist pitch (Arm 6). The 1-DoF human wrist flexion is mapped to the ORCA wrist, decoupled from the arm IK. A `--lock` flag fixes Arm 7 (6-DoF arm + ORCA wrist). The differential-IK formulation and singularity handling are derived in `whole_body_control.pdf`; this doc only states the retargeting cost.

2.4 Whole-body cascade (Layer 2)

The base-engage criterion is an asymmetric, front-of-head *bounding box* on the operator's HEAD displacement $\Delta = A(p_{\text{head}} - p_{\text{head},0})$, decomposed into a forward component $f = -\Delta_x$ (robot front $= -x$) and a lateral $\ell = \Delta_y$. The part of the offset *inside* the box, $f_{\text{in}} = \text{clip}(f, -b_{\text{back}}, b_{\text{fwd}})$, is absorbed by the **torso** (it bends $\propto f_{\text{in}}$, the fast first responder); only the *overflow* $f - f_{\text{in}}$ drives the **base**, with a position target $\propto w\,(f - f_{\text{in}})$. The box is deep in front ($b_{\text{fwd}} = 0.18$ m, so reaching/leaning forward a little uses torso + lift, not the wheels) and shallow behind ($b_{\text{back}} = 0.06$ m); per-direction weights ($w_{\text{back}} > w_{\text{fwd}}$) make backing up the most responsive. These are pure sensitivity *gains*, not separate objectives. The **lift** follows head height (speed-limited), and the operator's body heading drives the base yaw. See `whole_body_control.pdf`.

2.5 Base motion model and reverse guard (Layer 1)

The base velocity target is realized through MABEL's tip-over model (`motion_model.pdf`): the safe speed and acceleration shrink as $\sim 1/h$ with the lift-raised CoM height h, enforced by a support-polygon (ZMP) criterion. A *reference-governor* reverse guard additionally scales backward velocity to zero as the lift rises, because the single-rear-wheel base tips over its short front lever when braking/reversing while raised (Sec. 3).

2.6 Fail-safe engagement (Layer 4)

`EngagementSupervisor` watches the freshness of the stream, head, hands, and elbows and drives one rate-limited blend scalar $\alpha \in [0, 1]$ applied to the upper-body position actuators and the lift:

$$\text{ctrl} = (1 - \alpha)\,\text{home} + \alpha\,\text{live}. \tag{2}$$

On loss (headset removed $\Rightarrow$ stream stalls, or a limb drops out) α ramps to 0 over T_{retreat} and the upper body eases to a safe HOME pose (rest arms, lift lowered) while the base is frozen; on re-acquire α ramps to 1 over T_{engage} with `live` = the operator's *current* pose, and the world map (1) is re-anchored on the rising edge so there is no jump.

2.7 Clutch / re-indexing

`ClutchController` detects a single-hand *double-pinch* on EITHER hand: with hysteresis on each hand's pinch state, a rising edge is one click, and two clicks on the same hand within a short window toggle the clutch (a cooldown blocks re-triggering, so one gesture is one toggle). A double-pinch is deliberate and far less twitchy than holding a pose, and a single pinch is left free for grasping. A toggle *freezes* the whole body (the retargeting and whole-body cascade are skipped, the base stopped, the supervisor held engaged); a second double-pinch *releases* and re-anchors (1) exactly as in Sec. 2.6, so the robot eases from its held pose to the operator's new pose. This re-indexes the operator into a comfortable region without moving the robot, the embodied analogue of lifting and recentering a mouse.

2.8 Arm self-collision barrier

The natural arm IK can drive a hand into the torso, head, or the other arm. We keep the cosine IK as the desired pose q_{des} and *project* it onto the collision-safe set with the shipped capsule guard (`self_collision.py`; capsules on the two forearms and hands versus torso/head/opposite arm, closest-point distances). For the surface gap $d(q)$ of a monitored pair with separation normal n and point Jacobians J_a, J_b, the discrete control-barrier constraint $-n^\top (J_a - J_b)\,\dot{q} \le \alpha_c\,(d - d_s)$ is assembled over both arms jointly and a one-step velocity QP is solved,

$$\min_{\dot{q}} \ \tfrac{1}{2}\left\|\dot{q} - \dot{q}_{\text{des}}\right\|^2 \quad \text{s.t.} \quad G\,\dot{q} \le h, \ \ \dot{q}_{\text{lo}} \le \dot{q} \le \dot{q}_{\text{hi}}, \qquad q_{\text{safe}} = q_{\text{cur}} + \dot{q}\,\Delta t, \tag{3}$$

with $\dot{q}_{\text{des}} = (q_{\text{des}} - q_{\text{cur}})/\Delta t$ at the VR-frame interval Δt (so the barrier gain α_c [1/s] is dimensionally correct). The barrier derivation is in `self_collision.pdf`; the QP lets the hand *slide* along the obstacle while tracking the target as closely as the barrier allows. It is disabled by default and enabled with `--collision`; when enabled it is a strict no-op (targets pass straight through, no QP, no velocity clamp) until a monitored gap drops below the influence distance, so it never slows free motion.

3 Experiments

All numbers below are regenerated by `scripts/tools/teleop_benchmark.py`, which drives the exact per-frame pipeline headless on a scripted operator (both hands tracing arcs while the body walks, leans, and raises the head) and runs targeted safety probes. Fig. 1 collects per-stage latency, the tip-safe base envelope, and the self-collision barrier.

Rates and latency. The control/physics loop runs at ≈ 500 Hz (0.56 ms/step with the lidar sensors disabled), well above a 200 Hz target. Per solve, hand retargeting is ≈ 2.7 ms to 3.7 ms and the arm IK ≈ 3.1 ms to 4.1 ms (both run twice per VR frame, ≈ 120 Hz aggregate); the self-collision QP adds only ≈ 0.45 ms. The hand solver is cost-capped (8 SLSQP iterations $\times$ 16 DoF), while the arm solver's cost is target-dependent (1 iteration on a warm, reachable target; ~ 15 on a hard one); the shared bottleneck is a full-model forward-kinematics call per cost evaluation.

Tracking accuracy. On reachable targets (forward-kinematics of random in-bounds poses, cold start) the arm IK contact residual is ≈ 30 mm (p50) / 60 mm (p95) — accurate for a cosine retargeter that deliberately trades exact contact for a natural elbow/orientation posture. The larger end-to-end residual during locomotion (≈ 200 mm) is base-follow lag plus the reach/walk scale composition, not IK error (Sec. 6).

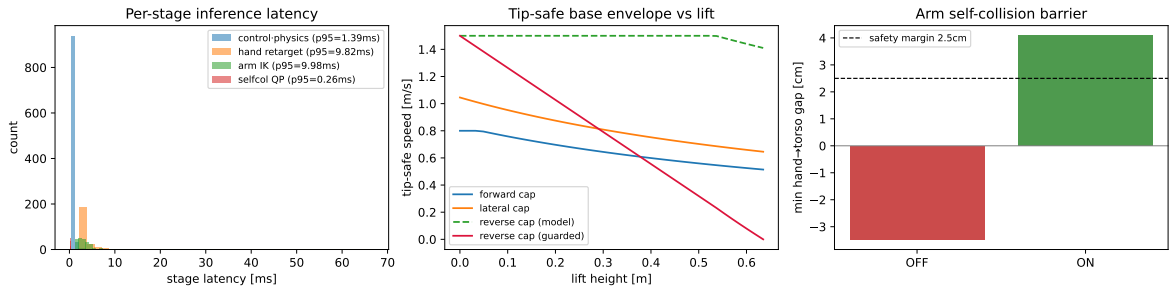


Figure 1: Left: per-stage inference latency (control/physics, hand retarget, arm IK, self-collision QP). Middle: tip-safe base speed envelope vs. lift height (forward, lateral, model reverse cap, and the guarded reverse cap that goes to zero raised). Right: minimum hand→torso gap on a march into the chest with the arm barrier OFF vs. ON, against the safety margin.

Locomotion stability. Across the reachable command envelope and lift heights $\{0, 0.32, 0.635\}$ m the maximum chassis tilt stays $< 4°$ (forward 0.51 m/s at full lift: 0.9°; strafe+turn: 3.8°); the reverse guard takes backward speed from 0.8 m/s (floor) to 0 m/s (full lift), all upright, where the unguarded model tipped.

Safety behaviour. The fail-safe eases to HOME on tracking loss ($T_{\text{retreat}} = 4$ s) and back on re-acquire ($T_{\text{engage}} = 2.5$ s), re-anchoring without a jump. The clutch toggles exactly once per held two-fist gesture and ignores a one-hand fist. On a hand marched into the chest, the arm barrier holds the gap at +4.8 cm (above the 2.5 cm margin) where the unguarded arm penetrated to −3.5 cm.

4 The live app path (Vision Pro / iOS / web over WebSocket)

The stack above is also the production controller for the apps: the WebSocket bridge feeds `teleop_frame` payloads (head + per-hand 27-joint ARKit skeletons as 4×4 world transforms, row-major) through `mabel.wire` into the same `TeleopEngine`. A July-2026 audit found the app path was *starved* of three inputs the desktop ZMQ path provides, which degraded it far below the documented behaviour; all three are now supplied server-side (no app changes):

1. **Full skeleton.** The wire ingest collapsed each hand to a 9-element wrist-padded stub (only thumb/index tips real); it now builds the full 21-landmark MediaPipe-layout list, so the palm frames and finger solvers see the real hand.

2. **Elbows.** ARKit provides no explicit elbow, so the arm IK's elbow-direction objective (the term that makes the redundant arm bend like the operator's) was silently off. The wire now derives each elbow from the headset's own `forearm_arm` joint (the elbow end of the ARKit forearm), exactly like the desktop streamer's published elbows.

3. **Torso heading.** The desktop stream publishes `body_yaw` from the two-elbow line (head-yaw–decoupled); the app sent none, so base yaw-follow was dead. The engine now runs the same elbow-line estimator (`ElbowYawEstimator`, low-passed with the streamer's $\tau = 0.25$ s, hold-last on dropout) on the derived elbows, with a late-anchoring yaw origin in `BaseTracker` so a heading that first appears after calibration cannot command a spin.

Webcam-parity fingers. On the app path the finger solver is `teleop/hand_anatomical.py` — a line-faithful port of the control-center GUI's webcam hand retargeting (`bodyteleop.decomposeHand` anatomical decomposition, PIP $+\frac{1}{2}$ DIP tendon coupling, thumb opposition $= \arcsin(-\hat{d} \cdot \hat{z})$, calibrated ORCA ranges), so every input device maps hands identically. One deliberate convention change: the JS labels handedness on the MediaPipe (selfie-mirrored) convention, so its chirality sign is *flipped* for true ARKit skeletons; a degenerate palm holds the last pose instead of snapping open, and a light EMA ($\alpha = 0.45$) filters ARKit jitter.

Anchor robustness and the height contract. Two live-session fixes (2026-07-08): (i) *ARKit origin-jump guard* — when the immersive space opens, ARKit can relocalise its world origin moments after tracking reports normal (the head teleports ∼1.6 m); anchors latched before that (world map, lift `head_h0`, base box) went stale and mapped the hands sky-high while the lift ran to its top. Anchors now latch only after 0.4 s of steady head tracking (`HEAD_ANCHOR_HOLD`), a per-frame head displacement over 0.35 m (`HEAD_JUMP_M` — impossible for a human head at frame rate) re-anchors everything, and `recalibrate()` clears the lift origin too. (ii) *Vertical 1:1* — REACH_SCALE now scales only the HORIZONTAL head-relative offset; the head→wrist *vertical* offset maps 1:1, so hands raised to the operator's head always sit at the *robot's* head height (validated: 4 cm palm-to-head at reach 1.2, 18 cm at an extreme 2.5). Relatedly, the bridge `hello` now advertises `h264Port` only when the H.264 encoder is actually enabled (`MABEL_SIM_H264=1`) — it previously advertised a dead port and the headset's video chased it instead of dropping straight to UDP-JPEG (`server/tools/exp_video_probe.py` asserts every advertised transport delivers).

Validation. `scripts/tools/exp_vp_app_teleop.py` regenerates six experiments end-to-end through the real pipeline (`teleop_frame` → wire → facade → engine → MuJoCo), seeded and headless: (E1) the Python port against the *actual* control-center JS executed in node: max joint discrepancy 6.7×10^{-16} rad over 200 random hands (exact parity); (E2) wrist-orientation sweeps (Fig. 2): palm-axis tracking error 7–10° mean / 16–28° max over ±60° roll and ±45° pitch, with the jointly-optimized ORCA wrist spanning 1.02 rad — orientation is genuinely tracked, bounded by the anatomical Arm 7 clamp and the cosine weights; (E3) commanded anatomical curls transfer 1:1 (max error $<$ 2 mrad, monotonic); (E4) the app base chain is *bit-exact* with the keyboard teleop's `safe_base_step` call (Fig. 3; forward steady speed derates 0.60 → 0.48 m/s at full lift, guarded reverse 0.60 → 0.03 m/s); (E5) a +30° operator torso turn yaws the chassis +30.0° (0.0° error, `figures/vp_body_yaw.pdf`), while a within-box head lean moves the base 0.000 m (torso absorbs it) and an overflow lean engages it (0.13 m).

5 Implementation and usage

The code is a modular package `scripts/teleop/` with one responsibility per module — `hyperparams` (all tunables, the single place to tune), `geometry`, `hand`, `arm`, `collision`, `motion_model`, `wholebody`, `safety`, `hud`, `overlay`, `model_utils` — and a thin orchestrator `scripts/retarget_teleop.py` that builds the objects and runs the control loop. Run the publisher then the consumer: `python scripts/vision_pro_stream.py --ip <headset>` and `mjpython scripts/retarget_teleop.py` (add `--lock` for a 6-DoF arm, `--collision` to enable the arm barrier, which is off by default). A live ANSI dashboard shows each stage's rate and latency, the base/lift/torso state, the self-collision gap, and the engagement/clutch mode. The console ZMQ channel accepts `reset`/`stop`/`start`; `stop` parks the robot at HOME. Swap points: replace `BaseMotionModel` or pass a `base_filter` to `WholeBodyController` (e.g. a base control barrier); replace `HandOptimizer`/`ArmRetargeter`; pass

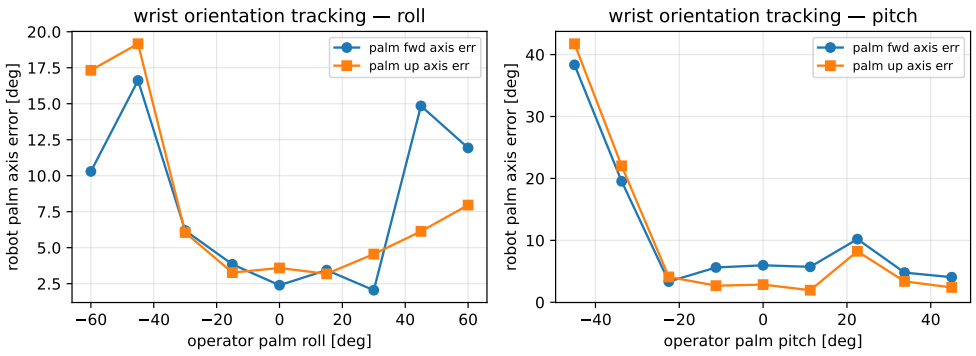


Figure 2: Wrist-orientation tracking through the full app pipeline: robot palm forward/up axis error over operator palm roll (left) and pitch (right) sweeps.

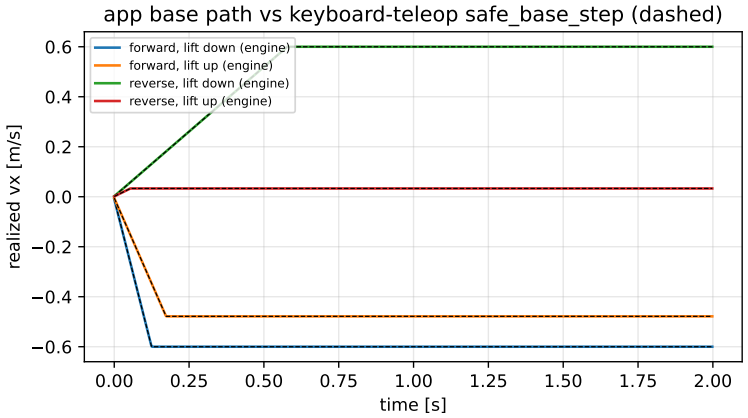


Figure 3: App base path vs. the keyboard-teleop reference chain (dashed): identical tip-safe ramps at both lift heights, including the lift-aware reverse guard.

a custom safety policy in place of `EngagementSupervisor`. Tune the arm-IK responsiveness with `ARM_W_SMOOTH` (lower = snappier), the barrier with `SELFCOL_INFLUENCE`/`SELFCOL_ALPHA`, and the clutch with the `CLUTCH_*` constants.

6 Limitations

(1) **Walk/reach composition:** (1) scales the operator's global walk by s while the base follows 1:1, so during locomotion the arm target outruns the base by $(s-1)\times$ walk plus the deadzone ($\approx 200\,\text{mm}$); a fix is to scale only the hand-relative component. (2) **Reverse at full lift** is handled by a guard, not by a fundamentally more stable base. (3) **Clutch trigger:** a both-hands double-pinch is deliberate and decoupled from single-pinch grasping, but a fast bimanual double-grasp could in principle register; the double-pinch window and cooldown set the sensitivity. (4) When enabled, the self-collision barrier slows both arms uniformly through the QP coupling (only while active) and guards only distal links (forearms/hands) versus torso/head/opposite arm. (5) No force/haptic feedback yet — the present loops close vision and motion, not touch. (6) **Open sim-base defect (2026-07-08):** the simulated swerve base realizes at most $\approx 0.042\,\text{m/s}$ regardless

of the commanded twist (84% of a 0.05 m/s command but only 5% of 0.8 m/s) — under combined drive the front wheels unload and spin while the rear wheel stalls at its 11 N m force limit. Verified NOT a controller defect (per-wheel steering, signs, and tip-safe commands are correct; each wheel driven alone moves the chassis correctly) and independent of `noslip_iterations`, drive force limits, and mesh-vs-sphere wheel collision geometry; it affects every simulated path identically (keyboard, app, studios) while the real robot realizes the same twist on hardware. The fix lives in the canonical MJCF (contact/compliance tuning) and must go through the model regen chain; base experiments should assert the wheel-actuator contract, not chassis displacement (see `server/tools/exp_vp_e2e_mock.py`).

References

[1] C. Ericson, *Real-Time Collision Detection*, Morgan Kaufmann, 2004 (segment–segment closest points, §5.1.9).

[2] E. Papadopoulos and D. Rey, "A new measure of tipover stability margin for mobile manipulators," *IEEE ICRA*, 1996.

[3] O. Khatib, "A unified approach for motion and force control of robot manipulators," *IEEE J. Robotics and Automation*, 1987 (operational space, null-space).

[4] A. D. Ames *et al.*, "Control barrier functions: theory and applications," *European Control Conference*, 2019.

[5] M. Schwarz *et al.*, "NimbRo Avatar: immersive bimanual humanoid telepresence," (ANA Avatar XPRIZE) — on whole-body avatar teleoperation and clutching.